

Proyecto Final de Base de Datos Avanzada: TraceFood.

Integrantes:

- Frandy Sena Taveras 2025-0945
- Wilcris Diaz Villanueva 2025-0911
- Sheyla De Leon 2025-0917
- Jambriel Calderon Ortiz 2025-0914

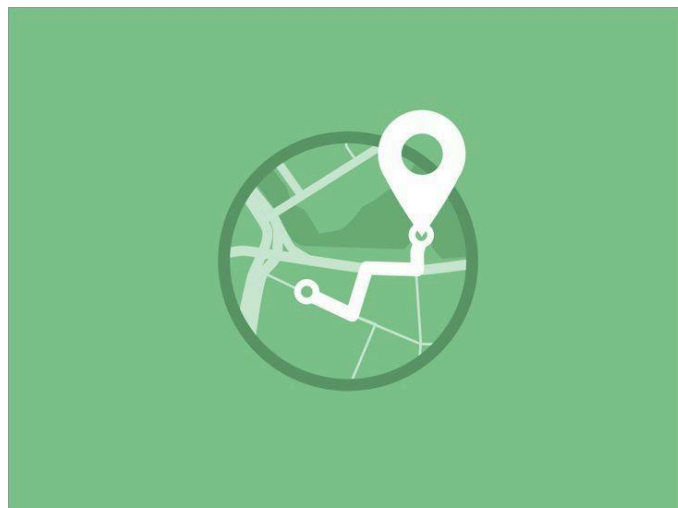
Maestro: Daniel Arturo Sánchez De Oleo

Introducción:

La eficiencia de un servicio de entrega no reside en la velocidad del motor, sino en la optimización de su trayecto. Presentamos una solución geoespacial diseñada para orquestrar la compleja relación entre locales y el trayecto que recorren los repartidores. Al integrar extensiones espaciales para el cálculo de polígonos de cobertura y rutas de entrega dinámicas, transformamos filas y columnas en una infraestructura resiliente capaz de mapear el apetito de una ciudad en tiempo real.

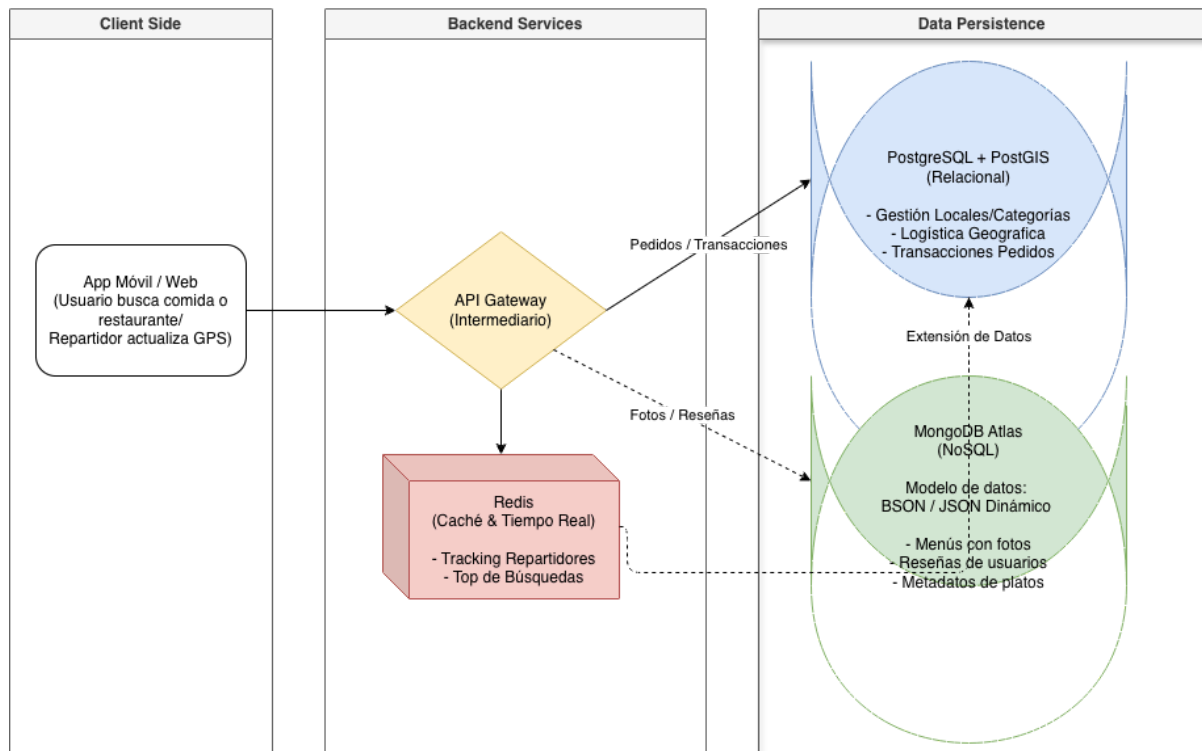
Este proyecto trasciende el simple almacenamiento de datos para convertir coordenadas en experiencias. Mediante una arquitectura híbrida que combina la precisión matemática de PostGIS, la flexibilidad documental de NoSQL y la velocidad efímera de Redis, hemos construido un ecosistema donde cada local no es solo un punto en el mapa, sino el epicentro de una decisión logística perfecta.

Porque en bases de datos geoespaciales, estar cerca no es suficiente; ser preciso es lo único que cuenta.



Arquitectura y descripción del proyecto:

El sistema se compone de un ecosistema híbrido donde conviven tres tecnologías de bases de datos distintas, orquestadas por un backend central. Como se puede apreciar en el diagrama de arquitectura adjunto a continuación, el flujo de información y responsabilidades se divide estratégicamente para aprovechar las fortalezas de cada paradigma:



1. Capa de Cliente (Client Side)

- **App Móvil / Web:** Es el punto de entrada de la aplicación. Es la interfaz donde el usuario final busca comida, gestiona su carrito y realiza órdenes. Asimismo, es la herramienta mediante la cual el repartidor actualiza constantemente su posición GPS durante el proceso de entrega.

2. Capa de Backend (API Gateway, Caché y Tiempo Real)

- **Tracking de Repartidores (Redis):** Las actualizaciones de ubicación frecuentes de los repartidores se procesan aquí primero. Al gestionarlas en memoria mediante Redis, evitamos saturar las secuencias BIGSERIAL y las operaciones de escritura en PostgreSQL, protegiendo el rendimiento de la base de datos principal.
- **Top de Búsquedas:** Almacena temporalmente los locales con más pedidos recientes para ofrecer recomendaciones instantáneas al cliente, reduciendo la carga de consultas pesadas hacia las bases de datos de disco duro.

3. Capa de Almacenamiento Relacional y No Relacional

Esta capa se divide en dos grandes paradigmas para ajustarse a la naturaleza de los datos del negocio:

A. PostgreSQL + PostGIS (Núcleo Relacional)

- **Gestión de Locales y Categorías:** Controla la relación e integridad referencial estricta entre los comercios y sus respectivas categorías de comida.
- **Logística Geográfica:** Almacenamiento preciso de coordenadas puntuales y el trazado del trayecto de los repartidores mediante objetos espaciales.
- **Transacciones:** Procesamiento robusto y seguro de los pedidos vinculados directamente a los usuarios y repartidores, garantizando que no existan fallos en los estados de compra.

B. NoSQL (MongoDB - Extensión de Datos)

- **Conexión Simbólica:** En el diseño lógico, esta base de datos se conecta de forma referencial siendo representada con una línea punteada en el diagrama, que conecta con la tabla de locales de PostgreSQL a través del ID del comercio.
- **Contenido:** Aquí viven los documentos dinámicos del sistema. Almacena los menús completos con fotos, las reseñas de texto largo dejadas por los usuarios y los metadatos variables de los platos que no encajan en una estructura de filas y columnas tradicional.

Base de Datos Relacional (PostgreSQL)

Creación de Tablas

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

```
-- 1. CREACIÓN DE TABLAS
```

```
-- =====
```

```
CREATE TABLE categories (  
  category_id BIGSERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  description TEXT  
);
```

```
CREATE TABLE stores (  
  store_id BIGSERIAL PRIMARY KEY,  
  category_id BIGINT NOT NULL REFERENCES categories(category_id),  
  name VARCHAR(150) NOT NULL,
```

```

    capacity INTEGER,
    coordinates GEOGRAPHY(Point, 4326) NOT NULL
);

CREATE TABLE delivery_zones (
    zone_id BIGSERIAL PRIMARY KEY,
    store_id BIGINT NOT NULL REFERENCES stores(store_id),
    zone_name VARCHAR(150) NOT NULL,
    coverage_polygon GEOMETRY(Polygon, 4326) NOT NULL
);

CREATE TABLE app_users (
    user_id BIGSERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(20),
    frequent_location GEOGRAPHY(Point, 4326)
);

CREATE TABLE drivers (
    driver_id BIGSERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    phone VARCHAR(20),
    current_location GEOGRAPHY(Point, 4326)
);

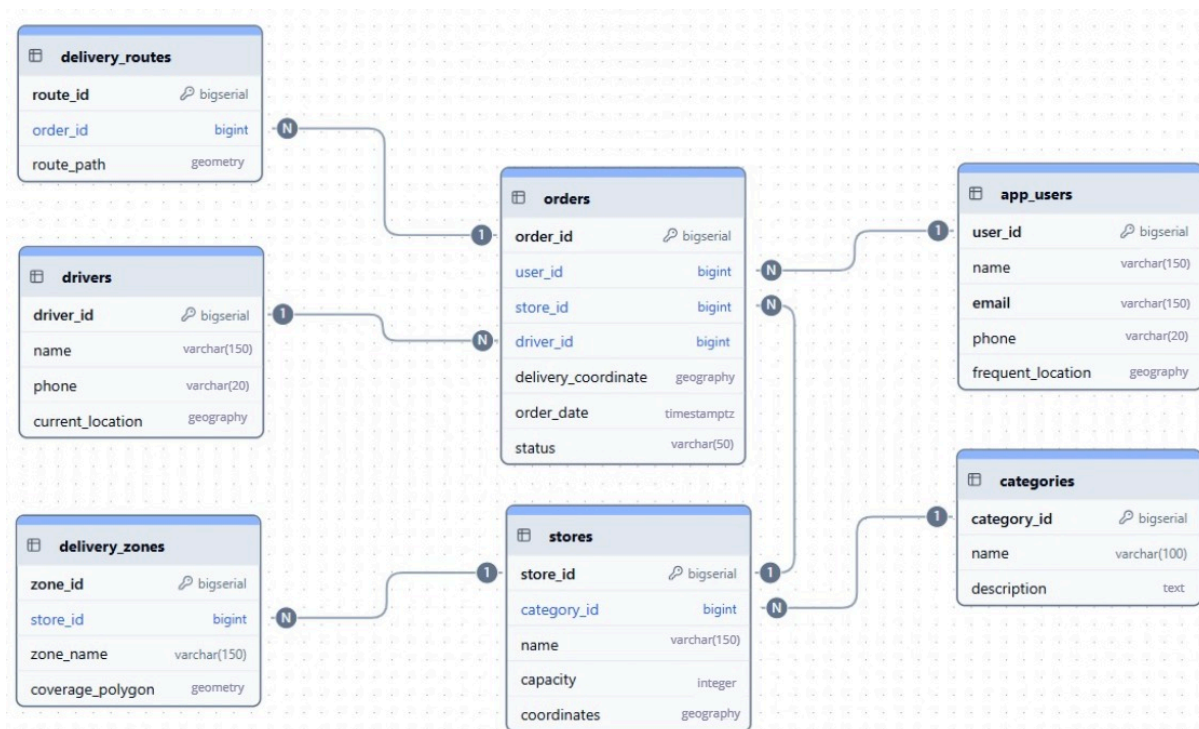
CREATE TABLE orders (
    order_id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES app_users(user_id),
    store_id BIGINT NOT NULL REFERENCES stores(store_id),
    driver_id BIGINT REFERENCES drivers(driver_id),
    delivery_coordinate GEOGRAPHY(Point, 4326) NOT NULL,
    order_date TIMESTAMPTZ DEFAULT NOW(),
    status VARCHAR(50) DEFAULT 'Pending'
);

CREATE TABLE delivery_routes (
    route_id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(order_id),
    route_path GEOMETRY(LineString, 4326) NOT NULL
);

```

Esta estructura representa el núcleo transaccional del sistema. Aquí se utiliza la extensión PostGIS para habilitar el soporte de objetos geográficos. En lugar de guardar latitud y longitud como simples flotantes, se utilizamos los tipos GEOGRAPHY y GEOMETRY con el sistema de referencia espacial SRID 4326. Esto permite que el motor de la base de datos calcule distancias reales en la curvatura de la Tierra y valide la contención de coordenadas dentro de polígonos de cobertura sin cálculos matemáticos pesados en el backend.

Diagrama de base de datos:



Inserción de Datos

Muestra representativa.

-- 1. Categories

INSERT INTO categories (name, description) VALUES

('Pica Pollo y Fritura', 'Pollo frito, tostones, batata y fritos verdes'),

('Comida Callejera', 'Chimis, yaroas, hot dogs y empanadas'),

('Comedores y Platos del Día', 'La bandera dominicana, sancocho y guarniciones'),

...

-- 2. Stores

INSERT INTO stores (category_id, name, capacity, coordinates) VALUES

-- Zona ITLA, Las Américas

(3, 'Comedor Doña Tata', 45, ST_SetSRID(ST_MakePoint(-69.6205, 18.4510),

4326)::geography),

...

-- 3. Delivery Zones

INSERT INTO delivery_zones (store_id, zone_name, coverage_polygon) VALUES

(1, 'Cobertura Comedor Campus', ST_GeomFromText('POLYGON((-69.64 18.44, -69.61 18.44, -69.61 18.47, -69.64 18.47, -69.64 18.44))', 4326)),

...

-- 4. App Users

```
INSERT INTO app_users (name, email, phone, frequent_location) VALUES
('Sheyla de Leon', 'sdeleon@campus.edu.do', '809-555-1001',
ST_SetSRID(ST_MakePoint(-69.6200, 18.4550), 4326)::geography),
```

...

-- 5. Drivers

```
INSERT INTO drivers (name, phone, current_location) VALUES
('Carlos Mendez', '829-555-9001', ST_SetSRID(ST_MakePoint(-69.6720, 18.4480),
4326)::geography),
```

...

-- 6. Orders

```
INSERT INTO orders (user_id, store_id, driver_id, delivery_coordinate, status) VALUES
(1, 1, 5, ST_SetSRID(ST_MakePoint(-69.6200, 18.4550), 4326)::geography, 'Delivered'),
```

...

-- 7. Delivery Routes

```
INSERT INTO delivery_routes (order_id, route_path) VALUES
(1, ST_GeomFromText('LINESTRING(-69.6205 18.4510, -69.6200 18.4530, -69.6200
18.4550)', 4326)),
```

...

Para poblar el sistema de manera realista, se utilizan funciones como ST_SetSRID y ST_MakePoint que transforman las coordenadas numéricas en entidades espaciales nativas válidas. Para los perímetros de entrega (coverage_polygon), se procesa un formato de texto estándar WKT mediante la función ST_GeomFromText, dibujando un recuadro o área específica alrededor del comercio.

Funciones, Procedimientos Almacenados, Triggers y Vistas

A. Funciones y Procedimientos Almacenados

muestra representativa.

-- 1. Calcular distancia lineal en metros entre el usuario y un local

```
CREATE FUNCTION calculate_user_store_distance(u_id BIGINT, s_id BIGINT)
RETURNS FLOAT AS $$
BEGIN
    RETURN ST_Distance(
        (SELECT frequent_location FROM app_users WHERE user_id = u_id),
        (SELECT coordinates FROM stores WHERE store_id = s_id)
    );
END;
```

```

$$ LANGUAGE plpgsql;

-- 2. Asignar el repartidor más cercano a un pedido específico
CREATE PROCEDURE assign_nearest_driver(p_order_id BIGINT)
AS $$
DECLARE
    v_store_coord GEOGRAPHY;
BEGIN
    SELECT s.coordinates INTO v_store_coord
    FROM stores s JOIN orders o ON s.store_id = o.store_id
    WHERE o.order_id = p_order_id;

    UPDATE orders
    SET driver_id = (
        SELECT driver_id FROM drivers
        ORDER BY current_location <-> v_store_coord LIMIT 1
    )
    WHERE order_id = p_order_id;
END;
$$ LANGUAGE plpgsql;

```

Las funciones encapsulan cálculos repetitivos que el backend consulta constantemente. Por ejemplo, `calculate_user_store_distance` usa `ST_Distance` para saber a qué distancia se encuentra el usuario de la tienda en línea recta. Por otro lado, el procedimiento `assign_nearest_driver` gestiona un proceso de negocio complejo de manera atómica: localiza la tienda de la orden y actualiza el pedido vinculando al repartidor más cercano mediante el operador `<->`, que ordena masivamente por cercanía de forma óptima.

B. Triggers

muestra representativa.

```

--2. Auto-asignar estado "En Camino" cuando se vincula un repartidor a un pedido
CREATE FUNCTION fn_update_order_status()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.driver_id IS DISTINCT FROM NEW.driver_id AND NEW.driver_id IS NOT NULL
    THEN
        NEW.status := 'On the Way';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tg_order_status_update ON orders;
CREATE TRIGGER tg_order_status_update
BEFORE UPDATE ON orders

```

```
FOR EACH ROW EXECUTE FUNCTION fn_update_order_status();
```

Los Triggers garantizan la automatización de procesos y el mantenimiento de la integridad en la base de datos sin depender del código de la aplicación. En este ejemplo, tan pronto como un repartidor es asignado a una orden en el campo `driver_id` pasa de ser nulo a tener un valor, el disparador cambia automáticamente el estado del pedido a *On the Way*, forzando el ciclo de vida correcto del negocio.

C. Vistas

muestra representativa.

--2. Vista de proximidad: Usuarios que tienen locales a menos de 2km de su casa

```
CREATE VIEW vw_users_with_nearby_stores AS
SELECT u.name AS user_name, s.name AS nearby_store,
       ST_Distance(u.frequent_location, s.coordinates) AS distance_meters
FROM app_users u, stores s
WHERE ST_DWithin(u.frequent_location, s.coordinates, 2000);
```

Las vistas exponen consultas complejas simulando ser tablas estáticas de solo lectura. En el caso de `vw_users_with_nearby_stores`, la vista precalcula qué clientes tienen restaurantes a un radio menor de 2 kilómetros de su casa. Esto simplifica enormemente el trabajo de los analistas de datos o del equipo de marketing para lanzar campañas hiperlocales.

Optimización y Consultas Avanzadas

muestra representativa.

-- ÍNDICES DE OPTIMIZACIÓN

-- =====

-- Los índices GIST son bastante importantes para hacer búsquedas rápidas con geometrías.

```
CREATE INDEX idx_stores_coordinates ON stores USING GIST (coordinates);
CREATE INDEX idx_users_location ON app_users USING GIST (frequent_location);
CREATE INDEX idx_orders_delivery ON orders USING GIST (delivery_coordinate);
CREATE INDEX idx_drivers_location ON drivers USING GIST (current_location);
CREATE INDEX idx_zones_polygon ON delivery_zones USING GIST (coverage_polygon);
CREATE INDEX idx_routes_path ON delivery_routes USING GIST (route_path);
```

-- Índices B-Tree normales para IDs y estados (aceleran los WHERE y JOINS)

```
CREATE INDEX idx_orders_status ON orders (status);
CREATE INDEX idx_orders_store_user ON orders (store_id, user_id);
```

-- CONSULTAS AVANZADAS Y MÉTRICAS


```
--
=====
===

-- Pedido más lejano de su tienda (Auditoría de distancias)
SELECT o.order_id, s.name AS store_name,
       ST_Distance(s.coordinates, o.delivery_coordinate) AS distance_m
FROM orders o
JOIN stores s ON o.store_id = s.store_id
ORDER BY distance_m DESC
LIMIT 10;
```

Para garantizar que el sistema no se congele cuando existan miles de usuarios y pedidos, se implementan dos tipos de índices:

1. **Índices GIST:** Imprescindibles en PostGIS. Permiten indexar geometrías en un árbol R para buscar rápidamente intersecciones y distancias sin recorrer toda la tabla secuencialmente.
2. **Índices B-Tree:** Usados en columnas tradicionales (como el estatus de la orden) para acelerar operaciones de ordenamiento (ORDER BY), agrupaciones (GROUP BY) y condicionales (WHERE).

Las consultas de auditoría como la anterior, permiten al negocio analizar si los repartidores están haciendo viajes demasiado largos que afecten el tiempo estimado de entrega.

Bases de Datos NoSQL (MongoDB)

1. Estrategia de Uso de MongoDB

En la aplicación de delivery, existen ciertos tipos de datos que no se adaptan eficientemente a un modelo relacional tradicional. Para estos casos, se implementa **MongoDB** como base de datos NoSQL, aprovechando su flexibilidad y alto rendimiento en escritura y lectura.

Utilizaremos MongoDB para los siguientes casos de uso:

- **Catálogos y Menús de Tiendas:** Los platos tienen atributos muy variables. Una pizza tiene mitades y tamaños, mientras que un Pica Pollo tiene piezas y acompañantes. MongoDB permite guardar el menú completo de un restaurante como un solo documento JSON, facilitando la lectura rápida en la app.
- **Recibos Históricos de Órdenes:** Cuando una orden se completa, los precios de los productos no deben cambiar en el recibo, incluso si el

restaurante actualiza su menú mañana. Se guardará un registro de la orden en MongoDB.

- **Reseñas y Calificaciones:** Los comentarios de los usuarios pueden incluir texto, fotos, puntuación del restaurante y puntuación del repartidor. Este formato semiestructurado es ideal para documentos NoSQL.

2. Diseño de Colecciones en MongoDB

Colección: menus

Almacena la oferta de cada restaurante. Se lee masivamente cuando un usuario abre el perfil de una tienda.

- **store_id:** Referencia al ID en PostgreSQL.
- **last_updated:** Fecha de la última modificación del menú.
- **categories:** Arreglo de categorías internas del restaurante (plato del día, bebida...)
- **items:** Arreglo de productos, cada uno con su nombre, precio, descripción, disponibilidad y modificadores u opciones personales.

Colección: order_receipts

Guarda el detalle inmutable de lo que el usuario compró, ideal para facturación y el historial de la app.

- **order_id:** Referencia a la tabla orders en PostgreSQL.
- **items_snapshot:** Arreglo con la copia exacta de los productos comprados, sus modificadores seleccionados y el precio al momento de la compra.
- **pricing:** Subtotal, impuestos, tarifa de envío (delivery fee) y total pagado.
- **payment_method:** Información de cómo se pagó

Colección: reviews

Almacena el feedback después de cada entrega

- **order_id:** Referencia a la orden completada.
- **user_id:** Referencia al usuario que comenta.
- **store_rating:** Puntuación del 1 al 5 para la comida.
- **driver_rating:** Puntuación del 1 al 5 para el repartidor.
- **comment:** Texto de la reseña.
- **photos:** Arreglo de URLs con imágenes subidas por el usuario.

3. Ejemplos de Documentos JSON

Ejemplo de Menú (Colección menus)

```
{
  "store_id": 2,
  "last_updated": "2026-03-26T08:00:00Z",
  "menu": [
    {
      "category": "Combos",
      "items": [
        {
          "item_id": "P001",
          "name": "Combo 3 Piezas",
          "description": "3 piezas de pollo frito tradicional dominicano.",
          "price": 250.00,
          "is_available": true,
          "modifiers": [
            {
              "name": "Guarnición",
              "required": true,
              "options": ["Tostones", "Fritos Verdes", "Batata Frita", "Papas Fritas"]
            }
          ]
        }
      ]
    }
  ]
}
```

codesnap.dev

Ejemplo de Recibo (Colección order_receipts)

```
{
  "order_id": 1,
  "user_id": 1,
  "store_id": 1,
  "completed_at": "2026-03-26T12:30:00Z",
  "items_snapshot": [
    {
      "name": "La Bandera Dominicana",
      "quantity": 1,
      "unit_price": 200.00,
      "selected_modifiers": ["Carne de Res", "Habichuelas Rojas"]
    },
    {
      "name": "Jugo de Chinola",
      "quantity": 1,
      "unit_price": 50.00,
      "selected_modifiers": ["Poca azúcar"]
    }
  ],
  "pricing": {
    "subtotal": 250.00,
    "delivery_fee": 50.00,
    "total": 300.00
  },
  "payment_method": "Efectivo"
}
```

codesnap.dev

Ejemplo de Reseña (colección reviews)

```
{
  "_id": { "$oid": "651a2b3c4d5e6f7a8b9c0d1e" },
  "order_id": 1,
  "user_id": 1,
  "store_id": 1,
  "driver_id": 5,
  "ratings": {
    "store": 5,
    "driver": 5
  },
  "feedback": {
    "store_comment": "¡La comida estaba excelente! La bandera dominicana tenía muy buen sabor, 100% recomendado para almorzar entre clases.",
    "driver_comment": "Alexander fue muy amable y llegó súper rápido a la entrada del ITLA."
  },
  "tags": ["Llegó caliente", "Buena porción", "Entrega rápida"],
  "photos": [
    "https://storage.tuapp.com.do/reviews/ord_1_foto1.jpg",
    "https://storage.tuapp.com.do/reviews/ord_1_foto2.jpg"
  ],
  "created_at": { "$date": "2026-03-26T13:45:00Z" }
}
```

codesnap.dev

Bases de Datos en Redis

Esta base de datos en Redis ha sido diseñada para almacenar y gestionar información proveniente de los sistemas principales, como PostgreSQL y MongoDB. Su propósito es optimizar el acceso a datos de alta demanda, facilitar operaciones en tiempo real y permitir el análisis eficiente de tendencias.

Gracias al uso de estructuras de datos nativas de Redis, se logran consultas super duper rápidas y flexibles, evitando sobrecargar los sistemas transaccionales y documentales.

Sus principales componentes:

1. Sesiones y Carrito de Compras

Los carritos de compra se almacenan mediante hashes, lo que permite mantener el estado temporal de los pedidos antes de que sean procesados y almacenados de forma definitiva en PostgreSQL.

Para evitar el consumo innecesario de memoria, cada carrito tiene un tiempo de expiración. Si el usuario abandona la aplicación o completa la compra, el carrito se elimina automáticamente.

Ejemplo:

```
HSET cart:user:1 store_id "3" item:P001:qty "2" subtotal "500.00"
```

Eliminación del carrito:

```
DEL cart:user:1
```

2. Ubicación de Repartidores en Tiempo Real

Para gestionar la ubicación de los repartidores, se utilizan estructuras geoespaciales mediante comandos como GEOADD, lo que permite actualizar y consultar posiciones en tiempo real con gran precisión.

Esto facilita encontrar rápidamente los conductores más cercanos a una tienda al momento de asignar pedidos.

Ejemplo:

```
GEOADD drivers:active_locations -69.6720 18.4480 "driver:5"
```

3. Caché de Menús de Alta Demanda

Los menús más consultados se almacenan como cadenas de texto en formato JSON, reduciendo la carga sobre MongoDB durante horas pico.

Estos datos se guardan con un tiempo de vida limitado por ejemplo, 1 hora, asegurando que la información se mantenga actualizada sin perder eficiencia.

Ejemplo:

```
SETEX cache:menu:store:1 3600 '{"store_id":1, "categories": [{"name":"Pica Pollo", "items": [...]}]'
```

4. Tendencias de Búsqueda y Tableros de Control

Las tendencias se gestionan mediante conjuntos ordenados, donde cada tienda incrementa su puntuación cada vez que recibe un pedido.

Esto permite generar rankings en tiempo real, ideales para mostrar los restaurantes más populares dentro de la aplicación.

Ejemplo:

ZINCRBY trending:stores:today 1 "store:3"

Consulta del Top 5:

ZREVRANGE trending:stores:today 0 4 WITHSCORES

5. Autocompletado de Búsqueda

Para mejorar la experiencia del usuario, se implementa un sistema de autocompletado utilizando conjuntos ordenados organizados por las letras.

Esto permite ofrecer sugerencias instantáneas mientras el usuario escribe en el buscador.

Ejemplo:

ZADD search:autocomplete 0 "pica pollo el expreso"

ZADD search:autocomplete 0 "pizzeria la abundancia"

6. Limitador de Peticiones

El control de peticiones se implementa mediante contadores atómicos con expiración, lo que permite proteger endpoints sensibles, como inicios de sesión o uso de cupones.

Por ejemplo, se puede limitar a un usuario a cierto número de intentos por minuto.

Ejemplo:

INCR rate_limit:coupons:user:12

EXPIRE rate_limit:coupons:user:12 60

Conclusiones

¿Por qué se eligió cada tecnología?

El núcleo de nuestra solución no reside en un único motor, sino en la sinergia de tres tecnologías líderes, seleccionadas estratégicamente para maximizar la disponibilidad, escalabilidad y velocidad de la plataforma de "Geo-Restaurantes". Nuestro proyecto busca crear un sistema capaz de recomendarte dónde comer basándose puramente en tu ubicación en tiempo real.

Si bien es cierto que dependemos de datos estructurados como las coordenadas, los nombres de los restaurantes y las categorías; también dependemos de datos "pesados" de lectura masiva como las galerías de fotos, las reseñas de los usuarios y los diferentes menús digitales. A esto le sumamos la necesidad de gestionar datos efímeros de altísima frecuencia, como la ubicación exacta del repartidor en ese preciso momento.

Para soportar esta complejidad, se implementó una arquitectura de persistencia políglota:

1. PostgreSQL + PostGIS: La Veracidad Geográfica

Elegimos PostgreSQL como el motor relacional e hilo conductor de todo el sistema debido a su robusta extensión PostGIS.

- **Precisión Matemática:** Permite manejar tipos de datos complejos como GEOMETRY y GEOGRAPHY para definir con exactitud milimétrica el polígono de cobertura de las zonas de entrega.
- **Integridad Referencial:** Garantiza bajo el estándar ACID que cada pedido esté vinculado directamente y sin fallos a un usuario y a un local, manteniendo la coherencia de los datos más críticos del negocio.

2. MongoDB: Flexibilidad Documental (NoSQL)

Mientras el mundo SQL gestiona la estructura rígida de las órdenes, MongoDB gestiona la experiencia dinámica del usuario.

- **Escalabilidad Horizontal:** Es el motor ideal para almacenar las reseñas de usuarios, galerías de fotos y menús digitales. Estos elementos poseen estructuras muy variables que saturaría y volverían lenta una tabla relacional tradicional.
- **Desacoplamiento:** Permite que la información multimedia y los catálogos crezcan exponencialmente sin afectar jamás el rendimiento de las consultas espaciales en la base de datos principal.

3. Redis: Latencia Cero en Tiempo Real

Para la logística dinámica y de alta velocidad, implementamos Redis como una capa pura de almacenamiento en memoria.

- **Tracking de Alta Frecuencia:** Las actualizaciones constantes de la ubicación_actual de los repartidores se procesan en Redis. Esto evita escrituras excesivas y bloqueos en el disco duro de PostgreSQL, garantizando una visualización fluida del repartidor moviéndose en el mapa del usuario.
- **Caché Inteligente:** Almacena temporalmente los resultados de las consultas más frecuentes (como el "Top 5 Restaurantes") para reducir drásticamente la carga de procesamiento sobre los servidores principales.